

CanlıFal — Kimlik Doğrulama Sistemi (Flutter Sözleşmesi)

Kaynak: `lib/mobile-auth.ts`, `lib/auth-options.ts`, `lib/rbac.ts`, `lib/permissions.ts`, `middleware.ts`

1. Çift kimlik mimarisi

Backend her korumalı route'ta **iki** yöntemi birden kabul eder:

Yöntem	Kim kullanır	Nasıl
mobile-jwt	Flutter / mobil	Authorization: Bearer <accessToken>
web-session	Tarayıcı	HTTP-only oturum çerezi

Route içindeki tipik kod:

```
const mobileUser = await authenticateRequest(req) // Bearer JWT
const session = await getServerSession(authOptions) // web çerezi
const userId = mobileUser?.id || session?.user?.id
if (!userId) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
```

Flutter için sonuç: tüm isteklere sadece `Authorization: Bearer <accessToken>` başlığı eklemek yeterlidir. Çerez yönetimi gerekmez.

2. Token modeli

`lib/mobile-auth.ts` :

Token	Geçerlilik	İçerik (payload)
accessToken	7 gün	{ userId, email, role, type: 'access' }
refreshToken	30 gün	{ userId, email, role, type: 'refresh' }

- İmza algoritması: HS256, sunucu tarafı gizli anahtar ile (istemciye verilmez).
- Doğrulama: `verifyMobileToken(token) → type === 'access'` olmayan token korumalı route'larda reddedilir.
- Performans: doğrulanan token'lar `lib/perf.ts` içindeki `getCachedAuth / setCachedAuth` ile kısa süreli önbellege alınır.

3. Giriş / kayıt endpoint'leri

Method	Path	Body	Dönen
POST	/api/auth/mobile-register	kayıt alanları	{ accessToken, refreshToken, user }
POST	/api/auth/mobile-login	{ email, password } (email alanı kullanıcı adı da kabul eder)	{ accessToken, refreshToken, user }
POST	/api/auth/mobile-refresh	{ refreshToken }	{ accessToken, refreshToken, user }
POST	/api/auth/mobile-google	Google id token	{ accessToken, refreshToken, user }
POST	/api/auth/mobile-apple	Apple identity token	{ accessToken, refreshToken, user }
POST	/api/auth/mobile-tiktok	TikTok auth kodu	{ accessToken, refreshToken, user }
POST	/api/auth/logout	—	oturum sonlandırma
POST	/api/auth/change-password	eski/yeni şifre	—
POST	/api/auth/forgot-password	{ email }	—
POST	/api/auth/reset-password	token + yeni şifre	—
POST	/api/auth/verify-device	cihaz doğrulama	—
POST	/api/auth/reclaim-device	cihaz geri alma	—

user nesnesi: { id, name, email, role, image, credits, jetonBalance, membership }

Rate limit: mobile-login IP başına authLimiter ile sınırlıdır → aşımında **429**.

4. Hata kodları

Kod	Anlamı	Flutter davranışı
400	Eksik/hatalı alan	Formu uyar
401	Token yok / geçersiz / süresi dolmuş	mobile-refresh dene, o da 401 ise çıkış yap
403	Yetki yetersiz (rol/izin)	Ekranı gizle
409	Kritik işlem onayı gerekli (\$88)	Aşağıya bakın
429	Hız sınırı	Geri çekilme (backoff) ile tekrar dene

409 — Kritik onay protokolü

Büyük tutarlı yönetici işlemlerinde sunucu şu gövdeyle **409** döner:

```
{ "requiresConfirmation": true, "confirmationMessage": "...", "action": "jeton_adjust" }
```

İstemci kullanıcıya mesajı gösterir ve **aynı gövdeyi** "confirm": true **ekleyerek** tekrar gönderir. Eşikler (lib/critical-confirm.ts): jeton ≥ 1000, CFC ≥ 5000, TL tutar ≥ 2000, kalıcı ban, rol değişimi, ödeme onay/düzeltilme/iade, ödül dağıtımı.

5. Roller ve yetki

lib/admin-utils.ts + lib/permissions.ts + lib/rbac.ts

- Roller: user , fortune_teller , agency , moderator , finans , yoneticici , admin
- Üyelik seviyeleri: basic , gold , diamond (+ membershipExpiresAt)
- resolveUser(req) (lib/rbac.ts) hem Bearer hem oturumu çözer; requireAuth , requireAdmin , requireFullAdmin , requireRole , requireOwnerOrAdmin yardımcıları vardır.
- İzin matrisi veritabanında tutulur: hasPermission , getRolePermissions , setRolePermissions .
- Yönetici sayfa route'ları middleware.ts ile korunur; **API route'ları middleware kapsamı dışındadır** (matcher api 'yi hariç tutar) ve her route kendi guard'ını çalıştırır.

6. API sürümlemesi

middleware.ts /api/v1/:path* isteklerini aynı handler'lara yeniden yazar ve x-api-version: v1 başlığı ekler.

Flutter istemcisi https://canlifal.com/api/v1/... tabanını güvenle kullanabilir.

7. Önerilen Flutter akışı

1. mobile-login → token'ları güvenli depoya yaz (flutter_secure_storage).

2. Dio/http interceptor: her isteğe `Authorization: Bearer` ekle.
3. 401 yakalandığında tek seferlik `mobile-refresh` ; başarılıysa isteği tekrarla, değilse oturumu kapat.
4. 429'da üstel geri çekilme.
5. 409 + `requiresConfirmation` → onay diyalogu → `confirm: true` ile tekrar gönder.